

A2 AP

fzt420, ltf688

September 2025

Indtroduction

The handed in solution consist of no failing test. The test are largely based on the assignment description itself. All requirements for the functionality of the code has should adhere to the task given by the assignment.

In implementation of `catch e1 e2`, we made a design choice *not* to forward the updated state from evaluating `e1` if it fails. This was not specified in the assignment, but seemed like the most logical choice.

Q1

Since we append each new printed string to the end of the string state, the time complexity will be Θn^2 , for n evaluations of print statements. Since Haskell is an immutable language, the entire list is copied for each call, and its size also increases with each call, thus giving us quadratic growth. This could be improved by instead prepending to the list, as that is constant time, since we can reuse the old list, and just allocate space for the new entry with a pointer to the old list. To keep the list in the correct order, it should be reversed when calling `runEval`.

Q2

In our implementation `KvPut` uses a linked list to store the pairs. Adding a key with an existing binding will not delete the old binding, but simple place a new one at the front of the list. Haskell's `lookup` function is then used to retrieve keys, and will pick the first match, ensuring the newest binding is used. As such the memory usage increases by adding more bindings of the same key.

Q3

No they are not visible, as stated in the introduction we chose *not* to foward the state returned from evaluating `e1` if it fails. If we wanted those changes to be visible we could change the code like so, forwarding `state'` to `m2`:

```
catch :: EvalM a -> EvalM a -> EvalM a
catch (EvalM m1) (EvalM m2) = EvalM $ \env state ->
  case m1 env state of
    (state', Left _) -> m2 env state'
    (state', Right x) -> (state', Right x)
```